Empirical Market Sentiment Analysis & Asset Price Correlation Pipeline

Corrected specification + phased implementation plan

Corrections applied to the original blueprint

1. **News source:** yfinance's built-in news feed is unreliable for this purpose — it often returns very few headlines per ticker and coverage is inconsistent. **NewsAPI.org (free tier, 100 requests/day) or GNews API (free tier)** are used as the primary source instead; yfinance news is kept only as a fallback.
2. **Statistical honesty:** with 5-10 tickers over 7-30 days (roughly 50-300 data points), correlation and the logistic regression are likely to come back weak or statistically insignificant. This is expected and must be stated plainly in the README's Limitations section, not hidden or overstated.
3. **Scope trim under time pressure:** if the 1-2 week window gets tight, Section 7.3 (logistic regression, ROC-AUC, confusion matrix) is the first thing to cut. Correlation (7.1) and directional accuracy (7.2) already answer the core research question on their own.
4. **Log return edge case:** drop the first row per ticker (no `t-1` exists for day one) rather than letting a `NaN` silently propagate downstream.

Implementation plan — phased

Each phase has a clear deliverable and a "don't move on until" checkpoint, so you always know whether you're actually ready for the next phase.

Phase 0: Environment & validation spike (0.5–1 day)

Goal: de-risk the single biggest unknown before writing real pipeline code.

- Set up the project folder using the exact repo structure from Section 11
- Create a virtual environment, install: `yfinance`, `nltk`, `pandas`, `matplotlib`, `seaborn`, `scikit-learn`, `requests`
- Sign up for a free NewsAPI.org or GNews API key
- Write a **throwaway test script** (not part of the real pipeline) that fetches headlines for 2-3 tickers over a 7-day window and prints how many headlines come back per ticker per day

Checkpoint — do not proceed until: you've confirmed you're getting a reasonable number of headlines (ideally 3+ per ticker per day on average) for at least one news source. If NewsAPI's free tier is too thin, test GNews before writing any pipeline code around it. This is the one thing that can quietly break the entire project later, so it's worth confirming first.

Phase 1: Data ingestion (1.5–2 days)

Goal: build `src/data_fetcher.py` — reliable, reusable fetching for both news and prices.

- Implement price ingestion via `yfinance` (Section 3.2): OHLCV + adjusted close, daily interval, for your configured ticker basket
- Implement news ingestion via your chosen API (Section 3.1): title, publisher, timestamp (UTC), URL, associated ticker
- Implement the retry/backoff logic (Section 10.2) — even a simple exponential backoff wrapper around your API calls
- Save raw outputs exactly as specified: `data/raw/raw_news.csv`, `data/raw/raw_prices.csv`

Checkpoint: open both CSVs and manually sanity-check them — do the dates make sense, are tickers correctly tagged, is there at least some news volume for every ticker in your basket? If one ticker has almost no news, consider swapping it out now rather than discovering it later.

Phase 2: Text cleaning & time alignment (1–1.5 days)

Goal: build `src/text_processor.py`.

- Text cleaning (Section 4.1): strip HTML/entities/non-printable characters, **preserve punctuation and capitalization** (important — VADER uses these for intensity), deduplicate identical headlines across sources
- Timestamp normalization to local exchange time (EST for US tickers, IST for NSE/BSE)
- Implement the **non-trading-day shift rule** exactly as specced: news after market close or on a weekend/holiday maps to the next valid trading day. This is the most methodologically important detail in the whole project — get it right here, since every downstream statistic depends on it
- Daily aggregation: group same-day headlines per ticker

Checkpoint: pick one ticker and manually trace 3-4 individual headlines through the shift logic by hand — confirm a Friday-evening headline correctly lands on Monday, and a mid-day Tuesday headline stays on Tuesday. Catching a bug here now saves you from silently wrong results later.

Phase 3: Sentiment scoring (1 day)

Goal: build `src/sentiment_engine.py`.

- Run NLTK VADER's `SentimentIntensityAnalyzer` on every cleaned headline
- Extract positive/negative/neutral/compound scores per headline (Section 5.2)
- Compute the daily aggregated sentiment index per ticker per day: mean compound score and headline volume (Section 5.3)

- Apply the zero-news-day rule (Section 10.1): impute neutral score (0.0) and set a `no_news = True` flag

Checkpoint: spot-check 5-10 headlines against their VADER scores manually — does an obviously positive headline ("Company X beats earnings expectations") score positive, does an obviously negative one score negative? This is a fast, cheap sanity check that catches silent pipeline bugs (e.g. accidentally scoring the wrong column).

Phase 4: Financial analytics & feature engineering (1 day)

Goal: build `src/financial_analytics.py`.

- Daily log return and percentage return (Section 6.1–6.2), **dropping the first row per ticker** to avoid the `NaN` from a missing `t-1`
- Directional target variable (Y_t)
- Intraday volatility (σ_t)
- Merge sentiment data (Phase 3 output) with price/return data (Phase 1 output) into the master analytical table, exactly matching the schema in Section 9: `[Ticker, Date, Headline_Count, Mean_Sentiment, Sentiment_Class, Close_Price, Daily_Return_Pct, Price_Direction]`
- Save as `data/processed/master_sentiment_prices.csv`

Checkpoint: open the master table and confirm row counts make sense (roughly: number of tickers × number of trading days in your window), and that there are no unexpected `NaN`s outside of the intentional first-day drop.

Phase 5: Statistical analysis (1–1.5 days)

Goal: the actual empirical test — this is the core "finding" of the project.

- Pearson and Spearman correlation between daily mean sentiment and both same-day return and next-day return (Section 7.1)
- Directional accuracy test (Section 7.2): what percentage of positive-sentiment days saw a positive return
- **If time allows:** baseline logistic regression using mean sentiment + headline volume to predict direction, with precision/recall/F1/ROC-AUC (Section 7.3) — treat this as optional/stretch given the small sample size caveat
- Write down your actual numeric results in a scratch note as you go — you'll need these for the README's Key Findings section

Checkpoint: you should be able to state, in one sentence, what you found — e.g. "same-day sentiment showed a weak positive Spearman correlation ($\rho \approx 0.14$, not statistically significant at n=180) with same-day returns." A vague or unstated result at this point means you're not ready to move to visualization yet — pin down the actual numbers first.

Phase 6: Visualization (1–1.5 days)

Goal: build `src/visualizer.py`.

- Dual-axis time-series chart (price vs sentiment, Section 8.1)
- Scatter plot with OLS trend line (sentiment vs return, Section 8.2)
- Confusion matrix heatmap (Section 8.3) — only if you did the logistic regression in Phase 5
- Sentiment category donut chart (Section 8.4)
- Export all as high-resolution PNGs to `outputs/figures/`

Checkpoint: look at each chart as if you were a stranger seeing it for the first time — is the story it tells the same as the numeric result you wrote down in Phase 5? If a chart looks like it shows a strong pattern but your correlation number was weak, double check for a plotting bug (e.g. unintentional smoothing or axis scaling that visually exaggerates the relationship).

Phase 7: Testing, README, and packaging (1–1.5 days)

Goal: make the project reproducible and presentable — this phase matters as much as the analysis itself for a cold-email/portfolio purpose.

- Write at least a few basic tests in `tests/test_sentiment.py` (e.g. confirm VADER scoring returns expected sign on a couple of known example headlines, confirm the shift-rule function handles a weekend date correctly) — doesn't need to be exhaustive, just enough to show testing discipline
- Write `README.md` per Section 12: executive summary, architecture diagram (ASCII is fine), key findings section with your actual embedded charts and numbers, reproducibility instructions (exact terminal commands), and an honest Limitations & Future Scope section — explicitly mention the small-sample caveat and VADER-vs-FinBERT tradeoff here
- Add `requirements.txt`, `.gitignore`, `LICENSE`
- Final run-through: clone your own repo into a fresh folder and follow your own README instructions exactly, to confirm it actually reproduces from scratch

Checkpoint — final: someone with no context should be able to read your README top to bottom and understand the question, method, honest result, and limitations in under 5 minutes.

Total estimated time: ~8-11 days at 10-15 hrs/week, comfortably within your 1-2 week target

Phase summary table

Phase	Focus	Est. time
0	Environment + news source validation	0.5–1 day
1	Data ingestion	1.5–2 days
2	Text cleaning + time alignment	1–1.5 days
3	Sentiment scoring	1 day
4	Financial analytics + feature engineering	1 day
5	Statistical analysis	1–1.5 days
6	Visualization	1–1.5 days
7	Testing, README, packaging	1–1.5 days